

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN		CICLO 02-2026
	MATERIA	DESARROLLO DE SOFTWARE PARA MOVILES	GUIA DE LABORATORIO #3
	PRÁCTICA	CICLO DE VIDA DE UNA APP	

I. OBJETIVOS

- Conocer el concepto de Activity.
- Conocer el ciclo de vida de una aplicación Android.
- Crear una aplicación Android e identificar en qué momento se ejecutan las diferentes etapas del ciclo de vida.

II. INTRODUCCIÓN TEORICA

Una Activity o Actividad en una aplicación Android es una interfaz. Es el componente de una aplicación donde están presentes elementos de interfaz gráfica, es decir, botones, radio botones, listados, cajas de texto, entre otros, y donde el usuario puede interactuar con la aplicación haciendo acciones como, tomar una foto, enviar un correo, entre otros.

Un ciclo de vida es una serie de estados secuenciales por los que una Activity pasa desde que es creada hasta que es destruida y en los que en cada uno se ejecutan ciertas acciones necesarias para el correcto funcionamiento de la aplicación. Cuando un usuario navega por una app, sale de ella y vuelve a entrar, las instancias de las Activity pasan por diferentes estados de su ciclo de vida. La clase Activity posee una serie de callbacks (estos funcionan como triggers o disparadores) que permiten a la actividad saber si cambio de estado y en qué estado se encuentra, así que es posible conocer si la actividad se está creando, reanudando o destruyendo. Dentro de los métodos callback del ciclo de vida, se puede declarar el comportamiento que tendrá la actividad cuando el usuario la abandone y la reanude.

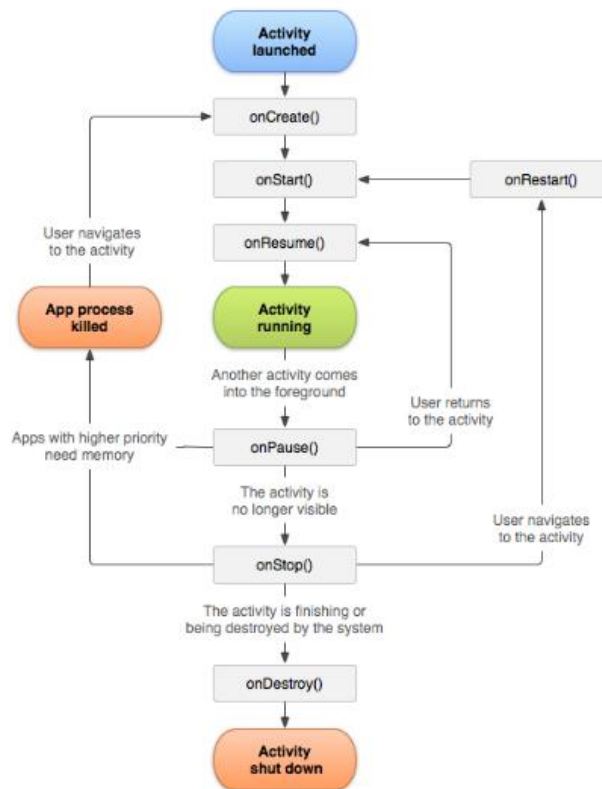
Por ejemplo, en caso de crear un reproductor de vídeo en streaming, se puede pausar el video y cancelar la conexión de red cuando el usuario cambia a otra app. Cuando el usuario vuelve, se puede volver a establecer la conexión con la red y permitir que el usuario reanude el video desde el mismo punto. En otras palabras, cada callback o devolución de llamada permite realizar un trabajo específico que es apropiado para un cambio de estado en particular. Hacer el trabajo preciso en el momento adecuado y administrar las transiciones correctamente hace que la app sea más sólida y eficiente.

Por ejemplo, una buena implementación de las devoluciones de llamada de un ciclo de vida puede ayudar a garantizar que la app:

- No falle si el usuario recibe una llamada telefónica o cambia a otra app mientras usa la aplicación desarrollada.
- No consuma recursos valiosos del sistema cuando el usuario no la use de forma activa.
- No pierda el progreso del usuario si este abandona la app y regresa a ella posteriormente.
- No falle ni pierda el progreso del usuario cuando se gire la pantalla entre la orientación horizontal y la vertical.

Para navegar por las transiciones entre las etapas del ciclo de vida de una actividad, la clase Activity proporciona un conjunto básico de seis métodos: **onCreate()**, **onStart()**, **onResume()**, **onPause()**, **onStop()** y **onDestroy()**. El sistema invoca cada uno de estos métodos cuando una operación entra en un nuevo estado.

A continuación, se muestra una representación visual de este paradigma.



onCreate()

Se debe implementar este método, que se activa cuando el sistema crea la actividad por primera vez. Cuando se crea la actividad, esta entra en el estado **Created**. En el método **onCreate()**, ejecuta la lógica de arranque básica de la aplicación **que debe ocurrir una sola vez en toda la vida de la actividad**. Generalmente este método es utilizado para inicializar la interfaz gráfica de la aplicación y/o preparar los datos que utilizas en la misma.

Es el lugar donde se incluye la mayor parte de la inicialización. Tareas prioritarias como:

- Poner atributos de diseño a la UI de la actividad con `setContentView()`.
- Agregar fragmentos.
- Obtener referencias de Views con `findViewById()`.
- Enlazar datos a los View.
- Iniciar consultas iniciales a fuentes de datos.
- Crear instancias de los componentes de tu arquitectura.

onStart()

La llamada **onStart()** hace que el usuario pueda ver la actividad mientras la app se prepara para que esta entre en primer plano y se convierta en interactiva. Por ejemplo, este método es donde la app inicializa el código que mantiene la IU (Interfaz de Usuario).

El método **onStart()** se completa muy rápido y, al igual que con el estado **Created**, la actividad no permanece en el estado **Started**. Una vez finalizada esta devolución de llamada, la actividad entra en el estado **Resumed**, y el sistema invoca el método **onResume()**.

onResume()

Cuando la actividad entra en el estado **Resumed**, pasa al primer plano y, a continuación, el sistema invoca la devolución de llamada **onResume()**. Este es el estado en el que la app interactúa con el usuario. La app permanece en este estado hasta que ocurre algún evento que la quita de foco. Tal evento podría ser, por ejemplo, recibir una llamada telefónica, que el usuario navegue a otra actividad o que se apague la pantalla del dispositivo.

Aquí es donde los componentes del ciclo de vida pueden habilitar cualquier funcionalidad que necesite ejecutarse mientras el componente esté visible y en primer plano, como, por ejemplo, iniciar una vista previa de la cámara. Cuando se produce un evento de interrupción, la actividad entra en el estado **Paused** y el sistema invoca la devolución de llamada **onPause()**. Si la actividad regresa al estado **Resumed** desde **Paused**, el sistema volverá a llamar al método **onResume()**.

onPause()

El sistema llama a este método a modo de primera indicación de que el usuario está abandonando tu actividad (aunque no siempre significa que está finalizando la actividad); esto indica que la actividad ya no está en primer plano (aunque puede seguir siendo visible si el usuario está en el modo multiventana). Utiliza el método **onPause()** para pausar o ajustar las operaciones que no deben continuar (o que deben continuar con moderación) mientras Activity se encuentra en estado **Paused** y que se espera reanudar en breve.

Cuando la actividad pase al estado de pausa, cualquier componente que priorice el ciclo de vida vinculado al ciclo de vida de la actividad recibirá el evento **ON_PAUSE**. Aquí es donde los componentes del ciclo de vida pueden detener cualquier funcionalidad que no necesite ejecutarse mientras el componente no esté en primer plano, como detener una vista previa de la cámara.

También se puede utilizar el método **onPause()** para liberar recursos del sistema, controladores de sensores (como el GPS) o cualquier otro recurso que pueda afectar la duración de la batería mientras tu actividad esté en pausa y el usuario no los necesite. Sin embargo, una actividad con el estado **Paused** puede ser completamente visible si está en el modo multiventana. Por eso, se debe considerar usar **onStop()** en lugar de **onPause()** para liberar o ajustar por completo los recursos y operaciones relacionados con la IU a fin de admitir mejor el modo multiventana.

La ejecución de **onPause()** es muy breve y no necesariamente permite disponer de tiempo suficiente para realizar operaciones seguras. Por esta razón, **no se debe utilizar onPause() para guardar los datos de la aplicación o del usuario, realizar llamadas de red o ejecutar transacciones de la base de datos, ya que es posible que no se complete dicho trabajo antes de que finalice el método**. En su lugar, se debe realizar operaciones de finalización de cargas pesadas durante **onStop()**.

La finalización del método **onPause()** no significa que la actividad abandone el estado **Paused**. Más bien, la actividad permanecerá en ese estado hasta que se reanude o se vuelva completamente invisible para el usuario. Si se reanuda la actividad, el sistema volverá a invocar la devolución de llamada **onResume()**. Si la actividad regresa del estado **Paused** a **Resumed**, el sistema mantendrá la instancia Activity en la memoria y la volverá a llamar cuando invoque **onResume()**. En esta situación, no es necesario que reinicialices los componentes que se crearon durante los métodos de devolución de llamada que llevan al estado **Resumed**. Si la actividad se vuelve completamente invisible, el sistema llamará a **onStop()**.

onStop()

Cuando el usuario ya no puede ver tu actividad, significa que ha entrado en el estado **Stopped**, y el sistema invoca la devolución de llamada **onStop()**. Esto puede ocurrir, por ejemplo, cuando una actividad recién lanzada cubre toda la pantalla. El sistema también puede llamar a **onStop()** cuando haya terminado la actividad y esté a punto de finalizar.

Cuando la actividad pase al estado **Stopped**, cualquier componente que priorice el ciclo de vida vinculado al de la actividad recibirá el evento **ON_STOP**. Aquí es donde los componentes del ciclo de vida pueden detener cualquier funcionalidad que no necesite ejecutarse mientras el componente no sea visible en la pantalla.

En el método **onStop()**, la app debe liberar o ajustar los recursos que no son necesarios mientras no sea visible para el usuario. Por ejemplo, la app podría pausar animaciones o cambiar de actualizaciones de ubicación detalladas a más generales. Usar **onStop()** en lugar de **onPause()** garantiza que continúe el trabajo relacionado con la IU, incluso cuando el usuario esté viendo tu actividad en el modo multiventana.

También se debe utilizar **onStop()** para realizar operaciones de finalización con un uso relativamente intensivo de la CPU. Por ejemplo, si no se encuentra un momento más oportuno para guardar información en una base de datos, se puede hacer en **onStop()**.

Desde el estado **Stopped**, la actividad regresa a interactuar con el usuario o se termina de ejecutar y desaparece. Si la actividad regresa, el sistema invoca a **onRestart()**. Si se terminó de ejecutar Activity, el sistema llamará a **onDestroy()**.

onDestroy()

Se llama a **onDestroy()** antes de que finalice la actividad. El sistema invoca esta devolución de llamada por los siguientes motivos:

- La actividad está terminando (debido a que el usuario la descarta por completo o a que se llama a **finish()**).
- El sistema está finalizando temporalmente la actividad debido a un cambio de configuración (como la rotación del dispositivo o el modo multiventana).

Cuando la actividad pase al estado **Destroyed**, cualquier componente que priorice el ciclo de vida vinculado al de la actividad recibirá el evento **ON_DESTROY**. Aquí es donde los componentes del ciclo de vida pueden recuperar cualquier elemento que se necesite antes de que finalice el objeto Activity.

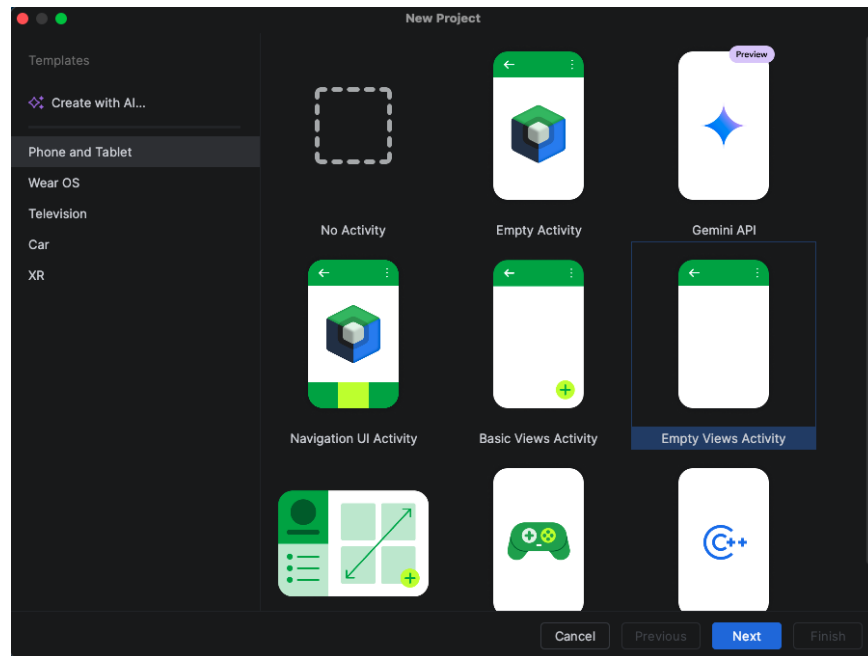
Si la actividad está terminando, **onDestroy()** es la devolución de llamada del ciclo de vida final que recibe la actividad. Si se llama a **onDestroy()** como resultado de un cambio de configuración, el sistema crea inmediatamente una nueva instancia de actividad y luego llama a **onCreate()** en esa nueva instancia en la nueva configuración.

La devolución de llamada **onDestroy()** debe liberar todos los recursos que aún no han sido liberados por devoluciones de llamada anteriores, como **onStop()**.

III. DESARROLLO DE LA PRÁCTICA

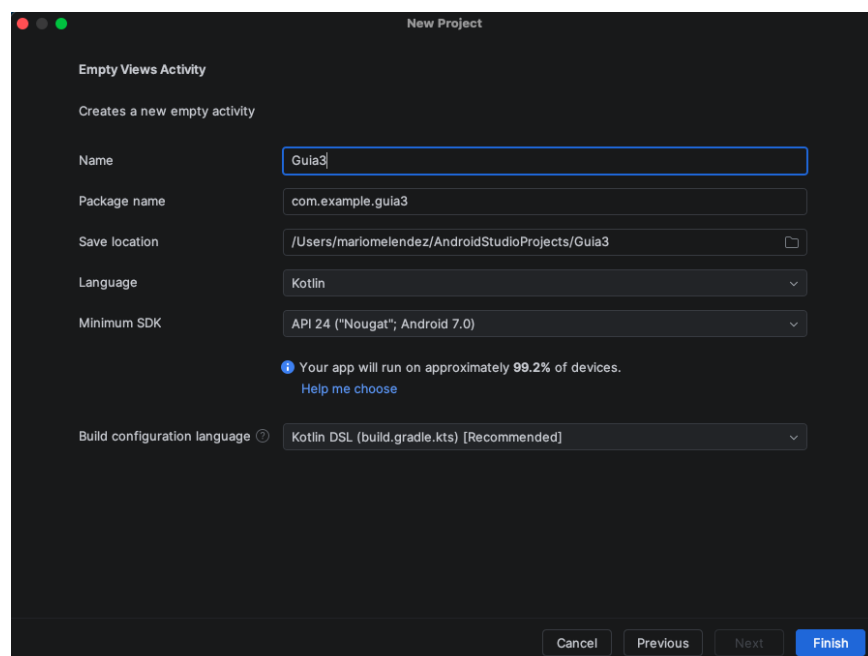
Parte 1 – LifeCycleObserver

1. Cree un nuevo proyecto en Android Studio, asegúrese de seleccionar la plantilla **Empty Views Activity**.

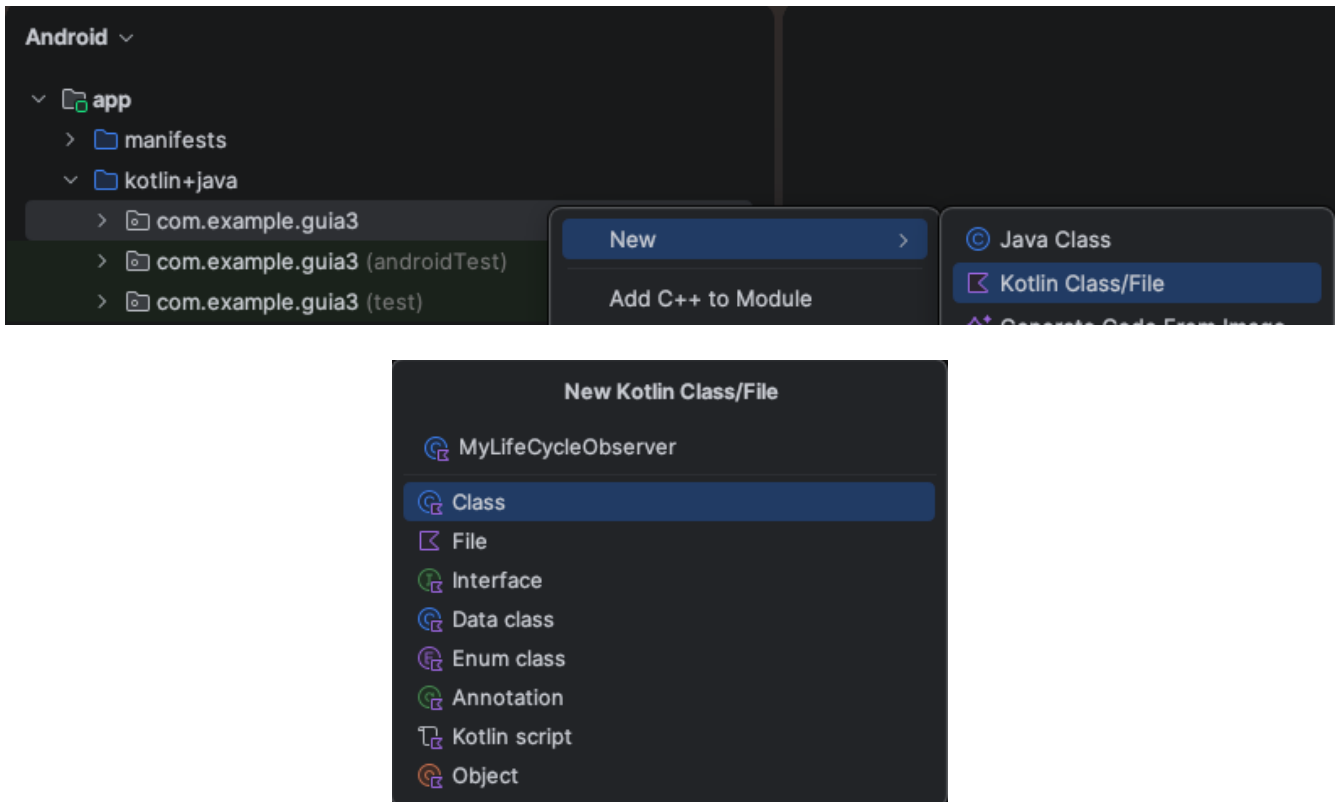


2. Configure el proyecto con las siguientes características:

- **Nombre:** Guia3.
- **Ubicación:** por defecto (o use una ubicación de su preferencia).
- **Lenguaje:** Kotlin.
- **Minimum SDK:** API 24 Nougat.



3. Una vez finalizada la sincronización del proyecto, sobre el paquete **com.example.guia3** (si usted colocó un nombre diferente al proyecto busque el paquete `com.example.<nombre>`) ubicado en la carpeta **kotlin+java**, agregue una nueva clase de kotlin con el nombre **MyLifecycleObserver**.



4. Modifique la clase **MyLifecycleObserver.kt** con el siguiente código:

```
package com.example.guia3

import android.util.Log
import androidx.lifecycle.DefaultLifecycleObserver
import androidx.lifecycle.LifecycleOwner

class MyLifecycleObserver : DefaultLifecycleObserver {
    override fun onCreate(owner: LifecycleOwner) {
        super.onCreate(owner)
        Log.d("MyLifecycleObserver", "onCreate")
    }

    override fun onStart(owner: LifecycleOwner) {
        super.onStart(owner)
        Log.d("MyLifecycleObserver", "onStart")
    }

    override fun onPause(owner: LifecycleOwner) {
```

```

        super.onPause(owner)
        Log.d("MyLifecycleObserver", "onPause")
    }

    override fun onResume(owner: LifecycleOwner) {
        super.onResume(owner)
        Log.d("MyLifecycleObserver", "onResume")
    }

    override fun onStop(owner: LifecycleOwner) {
        super.onStop(owner)
        Log.d("MyLifecycleObserver", "onStop")
    }

    override fun onDestroy(owner: LifecycleOwner) {
        super.onDestroy(owner)
        Log.d("MyLifecycleObserver", "onDestroy")
    }
}

```

- La clase implementa la interfaz **DefaultLifecycleObserver**, lo que le permite escuchar y responder a los cambios en el ciclo de vida de un componente de Android que implemente **LifecycleOwner**, como una Activity o un Fragment.

Esta interfaz proporciona métodos que corresponden a los distintos estados del ciclo de vida: **onCreate()**, **onStart()**, **onResume()**, **onPause()**, **onStop()**, y **onDestroy()**.

Cada uno de los métodos se ejecuta automáticamente cuando el componente observado y según sea el caso escribe en el log del sistema un mensaje descriptivo.

- Se utiliza la clase Log (Android.util.Log) para escribir mensaje en Logcat, **d()** es un método que escribe mensajes en el registro de tipo **Debug** que recibe dos parámetros:
 - **Tag:** que es una etiqueta con la cual podremos filtrar los registros.
 - **Message:** texto que queremos que se registre.

5. Modifique la clase **MainActivity.kt** con el siguiente código:

```

package com.example.guia3

import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    // 1. Referencia al archivo layout
    setContentView(R.layout.activity_main)
    // 2. Registro del observador
    lifecycle.addObserver(MyLifecycleObserver())
    mostrarToast("onCreate")
}

override fun onStart() {
    super.onStart()
    mostrarToast("onStart")
}

override fun onResume() {
    super.onResume()
    mostrarToast("onResume")
}

override fun onPause() {
    super.onPause()
    mostrarToast("onPause")
}

override fun onStop() {
    super.onStop()
    mostrarToast("onStop")
}

override fun onRestart() {
    super.onRestart()
    mostrarToast("onRestart")
}

override fun onDestroy() {
    super.onDestroy()
    mostrarToast("onDestroy")
}

private fun mostrarToast(
    mensaje: String,
    duracion: Int = Toast.LENGTH_SHORT
) {
    Toast.makeText(
        this,
        "MainActivity -> $mensaje",
        duracion
    )
}

```



```

        ).show()
    }
}

```

- Cada callback es sobrescrito utilizando la palabra reservada **override** para reemplazar el comportamiento original definido en la clase base AppCompatActivity. También, los callbacks llaman a la implementación de la superclase por medio **super** para ejecutar la lógica interna de la clase base.
- En el método **onCreate()** al ejecutarse cuando se crea la actividad por primera vez, se registra el observador para monitorear los cambios del ciclo de vida.
- Además de escribir logs en el registro del sistema, con **Toast** se muestran mensajes breves y flotantes para informar al usuario sobre el estado de la app.

6. Modifique el archivo **activity_main.xml** de la siguiente manera:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:background="#216EBC"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginVertical="20dp"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="#FFFFFF"
        android:text="Ciclo de Vida Android" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginVertical="20dp"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="#FFFFFF"
        android:text="Actividad principal" />

```

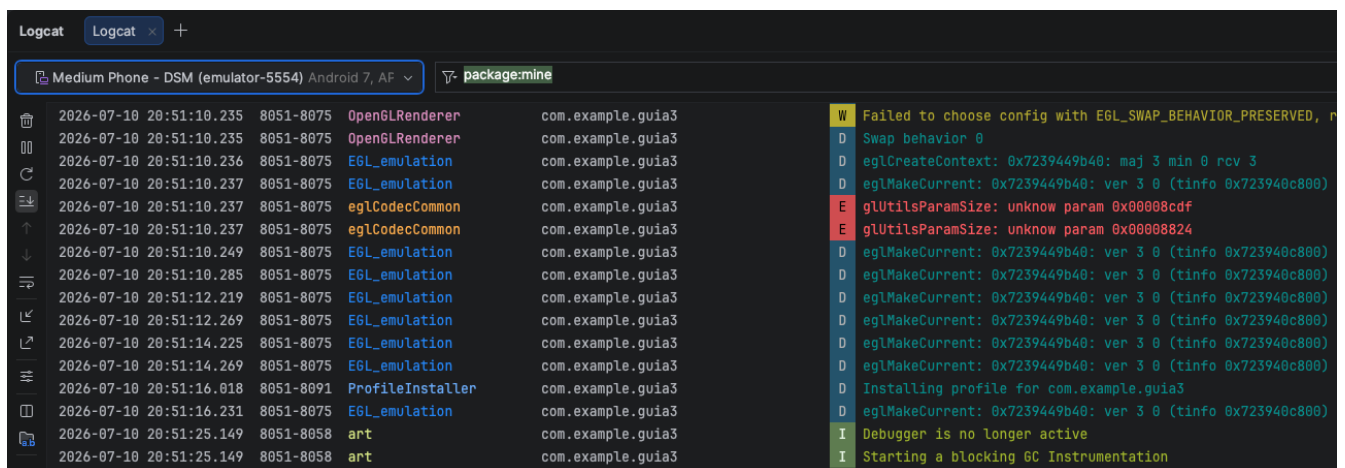
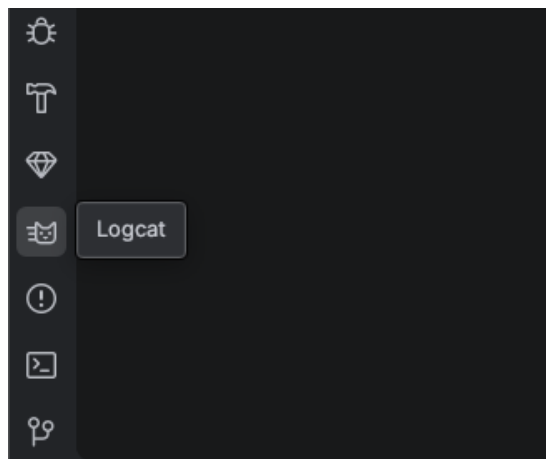
```
<!-- Boton para abrir nueva actividad -->
```

```
</LinearLayout>
```

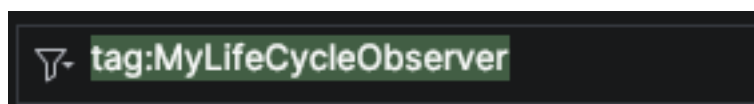
7. Ejecute la aplicación en el emulador o en un dispositivo móvil.

8. En la parte inferior izquierda de Android Studio ubique la herramienta **Logcat**, también puede ser encontrada en el menú principal **View > Tool Windows > Logcat**.

Logcat es una herramienta de línea de comandos y también una ventana en Android Studio que sirve para visualizar y depurar los registros del sistema Android, incluyendo mensajes generados por las aplicaciones y el propio sistema operativo.



9. Filtre los registros generados por la aplicación para demostrar el ciclo de vida nos ubíquese sobre el campo de filtro y escriba el criterio de búsqueda **tag:MyLifeCycleObserver**.



Medium Phone - DSM (emulator-5554) Android 7, AF tag:MyLifeCycleObserver

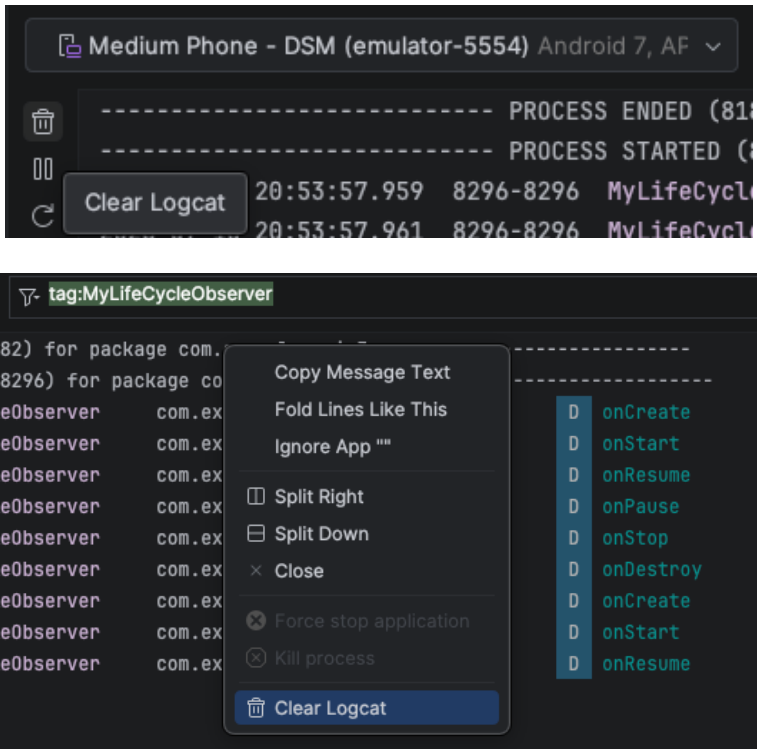
```

----- PROCESS ENDED (8182) for package com.example.guia3 -----
----- PROCESS STARTED (8296) for package com.example.guia3 -----
2026-07-10 20:53:57.959 8296-8296 MyLifeCycleObserver com.example.guia3 D onCreate
2026-07-10 20:53:57.961 8296-8296 MyLifeCycleObserver com.example.guia3 D onStart
2026-07-10 20:53:57.962 8296-8296 MyLifeCycleObserver com.example.guia3 D onResume
2026-07-10 20:54:02.135 8296-8296 MyLifeCycleObserver com.example.guia3 D onPause
2026-07-10 20:54:02.453 8296-8296 MyLifeCycleObserver com.example.guia3 D onStop
2026-07-10 20:54:02.454 8296-8296 MyLifeCycleObserver com.example.guia3 D onDestroy
2026-07-10 20:54:05.474 8296-8296 MyLifeCycleObserver com.example.guia3 D onCreate
2026-07-10 20:54:05.476 8296-8296 MyLifeCycleObserver com.example.guia3 D onStart
2026-07-10 20:54:05.477 8296-8296 MyLifeCycleObserver com.example.guia3 D onResume

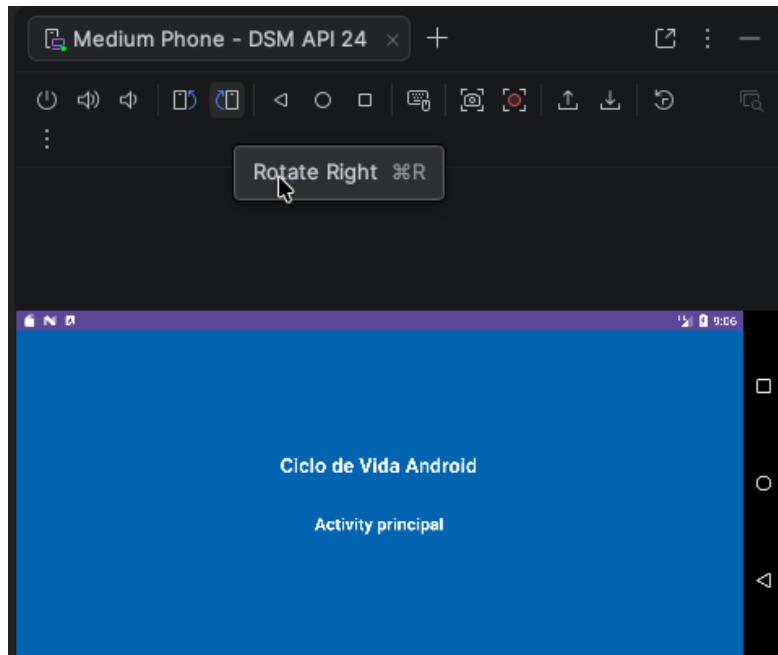
```

10. Realice al menos 5 diferentes interacciones (abrir una nueva aplicación, presionar el botón Home, Atrás) con el emulador o el dispositivo móvil y compruebe cuales son los mensajes que son visibles desde Logcat.

11. Limpie el log de registros, para ello haga clic en la opción **Clear Logcat** o sobre el panel de resultado haga clic derecho y seleccione la opción **Clear Logcat**.



11. Haciendo uso de los controles del emulador cambie la orientación del dispositivo (vertical a horizontal) y observe el resultado en el log.

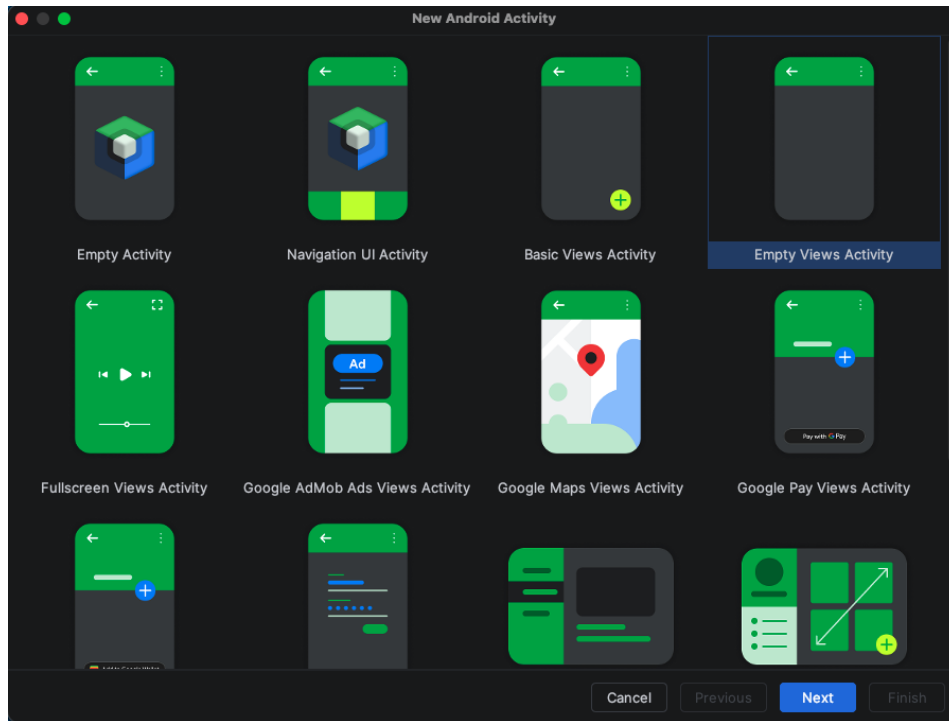


Medium Phone - DSM (emulator-5554) Android 7, AF				tag:MyLifecycleObserver
2026-07-10 21:05:30.435	8296-8296	MyLifecycleObserver	com.example.guia3	D onPause
2026-07-10 21:05:30.447	8296-8296	MyLifecycleObserver	com.example.guia3	D onStop
2026-07-10 21:05:30.458	8296-8296	MyLifecycleObserver	com.example.guia3	D onDestroy
2026-07-10 21:05:30.473	8296-8296	MyLifecycleObserver	com.example.guia3	D onCreate
2026-07-10 21:05:30.474	8296-8296	MyLifecycleObserver	com.example.guia3	D onStart
2026-07-10 21:05:30.478	8296-8296	MyLifecycleObserver	com.example.guia3	D onResume

- Los cambios de configuración como rotación de la pantalla, cambio de idioma o resolución de pantalla provocan que la Activity se destruya y recree nuevamente, es por eso por lo que el flujo del ciclo de vida pasa por los callbacks **onPause()** > **onStop()** > **onDestroy()** > **onCreate()** > **onStart()** > **onResume()**.

Parte 2 – Agregando una nueva Activity

1. Agregue una nueva Activity haciendo clic derecho sobre el proyecto y seleccionando la opción **New > Activity > Gallery**. De las plantillas seleccione la opción **Empty Views Activity** y haga clic en **Next**.



2. Configure la nueva Activity como se muestra a continuación, asegúrese de marcar la opción **Generar un archivo Layout (Generate a Layout File)**, luego haga clic en **Finish**.

Empty Views Activity
Creates a new empty activity

Activity Name
SecondActivity

☒ Generate a Layout File

Layout Name
activity_second

☐ Launcher Activity

Package name
com.example.guia3

Source Language
Kotlin

Target Source Set
main

Cancel Previous Next **Finish**

3. Modifique el contenido del archivo **activity_second.xml** de la siguiente forma.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:background="#009688"
tools:context=".SecondActivity">
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginVertical="20dp"
    android:textSize="24sp"
    android:textStyle="bold"
    android:textColor="#FFFFFF"
    android:text="Ciclo de Vida Android" />
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginVertical="20dp"
    android:textSize="20sp"
    android:textStyle="bold"
    android:textColor="#FFFFFF"
    android:text="Activity secundaria" />
<Button
    android:id="@+id/btnRegresar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Regresar a Main Activity"
    android:layout_marginVertical="20dp"
    android:textSize="20sp"
    android:textStyle="bold" />
</LinearLayout>

```

4. Modifique la clase **MyLifecycleObserver.kt** para agregar un parámetro de nombre **activity** y tipo **String** en el constructor primario de la siguiente manera.

```

class MyLifecycleObserver(val activity: String) : DefaultLifecycleObserver {
    /* Callbacks */
}

```

- El parámetro **activity** tendrá el nombre del activity que registra el observador para que se muestre en el log del sistema.

5. Modifique todos los métodos de la clase **MyLifeCycleObserver** para mostrar el valor enviado en el constructor. Tome el siguiente ejemplo y replíquelo en los métodos restantes de la clase.

```
override fun onCreate(owner: LifecycleOwner) {  
    super.onCreate(owner)  
    Log.d("MyLifeCycleObserver", "$activity -> onCreate")  
}
```

6. Modifique el código de la clase **SecondActivity** como se muestra a continuación.

```
package com.example.guia3  
  
import android.content.Intent  
import android.os.Bundle  
import android.widget.Button  
import android.widget.Toast  
import androidx.appcompat.app.AppCompatActivity  
  
class SecondActivity : AppCompatActivity() {  
  
    lateinit var btnRegresar: Button  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        // 1. Referencia al archivo layout  
        setContentView(R.layout.activity_second)  
        // 2. Referencia al boton  
        btnRegresar = findViewById(R.id.btnRegresar)  
        // 3. Registro del observador  
        lifecycle.addObserver(MyLifeCycleObserver("SecondActivity"))  
  
        // 4. Listener del boton  
        btnRegresar.setOnClickListener{  
            // 5. Uso de un intent explicito para iniciar una nueva Activity  
            val intent = Intent(this, MainActivity::class.java)  
            startActivity(intent)  
        }  
        mostrarToast("onCreate")  
    }  
  
    override fun onStart() {  
        super.onStart()  
        mostrarToast("onStart")  
    }  
}
```

```

override fun onResume() {
    super.onResume()
    mostrarToast("onResume")
}

override fun onPause() {
    super.onPause()
    mostrarToast("onPause")
}

override fun onStop() {
    super.onStop()
    mostrarToast("onStop")
}

override fun onRestart() {
    super.onRestart()
    mostrarToast("onRestart")
}

override fun onDestroy() {
    super.onDestroy()
    mostrarToast("onDestroy")
}

private fun mostrarToast(
    mensaje: String,
    duracion: Int = Toast.LENGTH_SHORT
) {
    Toast.makeText(
        this,
        "SecondActivity -> $mensaje",
        duracion
    ).show()
}
}

```

7. Modifique el archivo **activity_main.xml** para agregar en la sección **<!-- Boton para abrir nueva actividad -->** el botón encargado de abrir la actividad secundaria.

```

<!-- Botón para abrir nueva actividad -->

<Button
    android:id="@+id/btnAbrir"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"

```



```

    android:text="Abrir nueva Activity"
    android:layout_marginVertical="20dp"
    android:textSize="20sp"
    android:textStyle="bold" />

```

7. Corrija en la clase **MainActivity** el callback **onCreate** para enviar en el registro del observador el parámetro **"MainActivity"**, además, agregue el listener para abrir la actividad **SecondActivity**. Asegúrese de agregar antes del callback **onCreate** la variable **btnAbrir**.

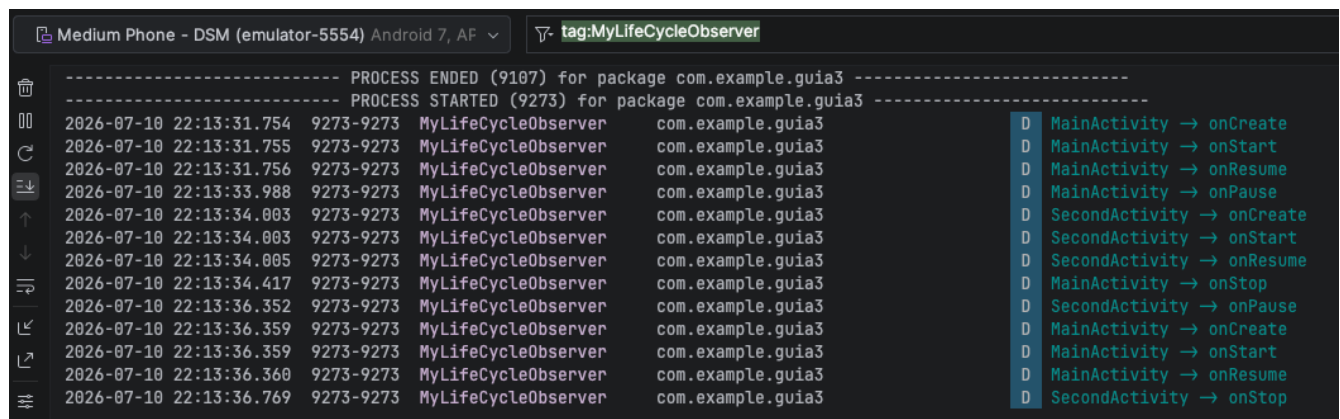
```

lateinit var btnAbrir: Button

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    /// 1. Referencia al archivo layout
    setContentView(R.layout.activity_main)
    // 2. Referencia al boton
    btnAbrir = findViewById(R.id.btnAbrir)
    // 3. Registro del observador
    lifecycle.addObserver(MyLifecycleObserver("MainActivity"))
    // 4. Listener del boton
    btnAbrir.setOnClickListener{
        // 5. Uso de un intent explicito para iniciar una nueva Activity
        val intent = Intent(this, SecondActivity::class.java)
        startActivity(intent)
    }
    mostrarToast("onCreate")
}

```

8. Ejecute y pruebe la aplicación, observe el resultado en el log cuando se abre una nueva actividad.



```

Medium Phone - DSM (emulator-5554) Android 7, AF  tag:MyLifecycleObserver

----- PROCESS ENDED (9107) for package com.example.guia3 -----
----- PROCESS STARTED (9273) for package com.example.guia3 -----
2026-07-10 22:13:31.754 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onCreate
2026-07-10 22:13:31.755 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onStart
2026-07-10 22:13:31.756 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onResume
2026-07-10 22:13:33.988 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onPause
2026-07-10 22:13:34.003 9273-9273 MyLifecycleObserver com.example.guia3 D SecondActivity → onCreate
2026-07-10 22:13:34.003 9273-9273 MyLifecycleObserver com.example.guia3 D SecondActivity → onStart
2026-07-10 22:13:34.005 9273-9273 MyLifecycleObserver com.example.guia3 D SecondActivity → onResume
2026-07-10 22:13:34.417 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onStop
2026-07-10 22:13:36.352 9273-9273 MyLifecycleObserver com.example.guia3 D SecondActivity → onPause
2026-07-10 22:13:36.359 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onCreate
2026-07-10 22:13:36.359 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onStart
2026-07-10 22:13:36.360 9273-9273 MyLifecycleObserver com.example.guia3 D MainActivity → onResume
2026-07-10 22:13:36.769 9273-9273 MyLifecycleObserver com.example.guia3 D SecondActivity → onStop

```

9. Realice la misma prueba de rotar la pantalla, compruebe cuales son los callbacks que se llaman en el ciclo de vida.

Parte 3 – Persistir estado de la aplicación

1. Modifique el archivo **activity_main.xml** y agregue los siguientes elementos al final del **LinearLayout**.

```
<!-- Contador -->
<Button
    android:id="@+id/btnIncrementar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Incrementar"
    android:backgroundTint="#E91E63"
    android:layout_marginTop="20dp"
    android:textSize="20sp"
    android:textStyle="bold" />

<TextView
    android:id="@+id/tvContador"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="20sp"
    android:textStyle="bold"
    android:textColor="#FFFFFF"
    android:text="Valor contador: 0" />
```

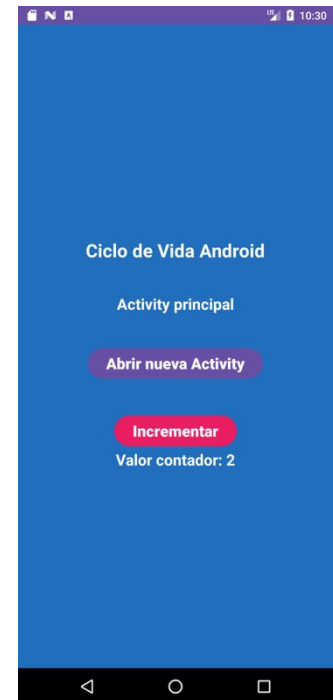
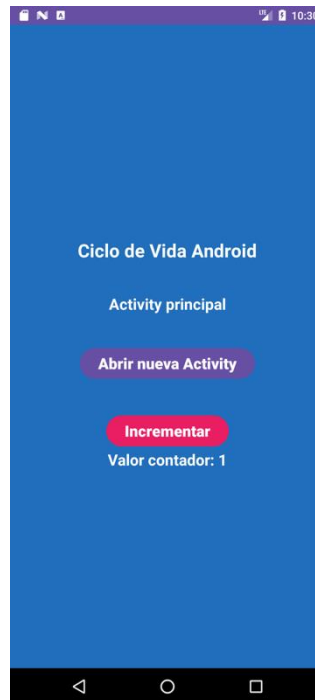
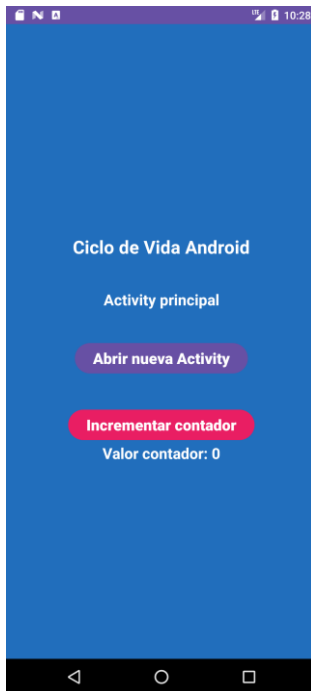
2. Modifique la clase **ActivityMain.kt** para agregar las variables que se muestran a continuación

```
lateinit var btnAbrir: Button
lateinit var btnIncrementar: Button // <- Nuevo
lateinit var tvContador: TextView // <- Nuevo
var contador: Int = 0 // <- Nuevo
```

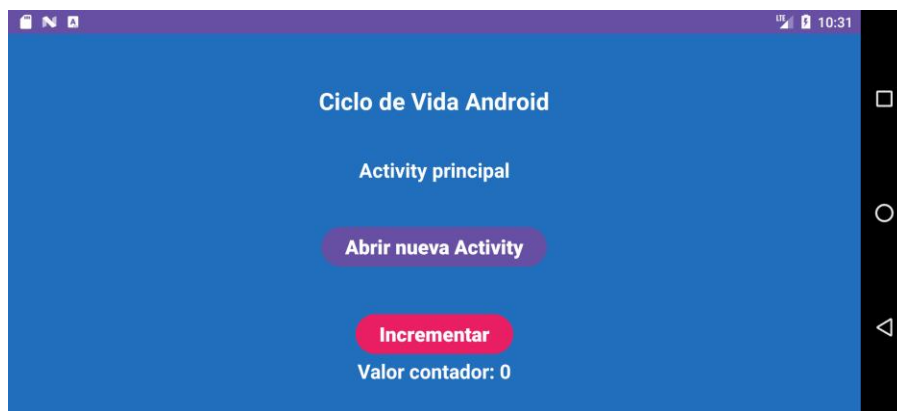
3. Modifique el callback onCreate de la clase **ActivityMain.kt** y agregue el listener del botón btnIncrementar de la siguiente manera.

```
// 6. Lister del botón incrementar
btnIncrementar.setOnClickListener{
    tvContador.text = "Valor contador: ${++contador}"
}
```

4. Ejecute la aplicación y compruebe el funcionamiento del botón **Incrementar**.



5. Cambie la orientación de la pantalla y compruebe el valor visible del contador.



- El valor de variable contador es inicializado en CERO debido a que la Activity fue destruida y recreada.

6. Modifique la clase **MainActivity** y agregue el siguiente callback al final.

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putInt("CONTADOR", contador)
}
```

- El método **onSaveInstanceState()** forma parte del ciclo de vida y se utiliza para guardar temporalmente el estado de la interfaz antes que Android destruya la Activity por un cambio de configuración o por una liberación de memoria.
- El estado/datos son guardados en un contenedor (Bundle) de tipo clave-valor.

- Al igual que los otros métodos del ciclo de vida se llama el método de la clase base con **super**.

7. Para recuperar el estado y lograr sobrevivir a la recreación de la actividad, modifique el callback onCreate de clase **MainActivity** de la siguiente manera.

```
override fun onCreate(savedInstanceState: Bundle?) {
    /* codigo anterior */

    // 7. Recuperar estado
    if (savedInstanceState != null) {
        contador = savedInstanceState.getInt("CONTADOR")
        tvContador.text = "Valor contador: ${contador}"
    }

    mostrarToast("onCreate")
}
```

8. Compruebe el funcionamiento de la aplicación, verifique que el contador no se reinicia a pesar del cambio de configuración.



IV. DISCUSIÓN DE RESULTADOS

Ejercicio 1

1. En el mismo proyecto agregue una nueva actividad al proyecto con el nombre **ThirdActivity**.
2. Configure la clase **ThirdActivity** para registrar adecuadamente su ciclo de vida y el Toast correspondiente.
3. Modifique el nuevo layout generado para identificar visualmente que es un nuevo Activity.
4. Haga los cambios en las clases **MainActivity** y **SecondActivity**, así como en **activity_main.xml** y **activity_second.xml** para permitir navegar entre las 3 actividades.

Ejercicio 2

Desarrollar una aplicación Android que permita capturar información personal en una pantalla y conservar el estado de la aplicación ante cambios de configuración utilizando el ciclo de vida de las Activities.

1. La primera pantalla deberá contener un formulario con los siguientes campos: nombres, apellidos, correo electrónico, teléfono celular, edad (numérica).
2. Incluya un botón con el texto "Validar información", donde, al ser presionado se valide que todos los campos estén completos y, además, compruebe si la persona es mayor de edad o no.
3. Muestre un resumen de la información ingresada y de los resultados obtenidos dentro de la misma Activity utilizando un TextView.
4. La aplicación deberá conservar toda la información ingresada y el resultado mostrado cuando ocurra un cambio de configuración (al rotar el dispositivo o recrearse la Activity).

V. BIBLOGRAFÍA

- Documentación Oficial Android.
Cómo interpretar el ciclo de vida de una actividad
<https://developer.android.com/guide/components/activities/activity-lifecycle#es-419>